

Universidad de Oriente	
Machine Learning	Clase 10 - Ejemplo 2 - Regresión de Ridge, Lasso y Red Elástica

```
In [ ]: import pandas as pd
import os
import tarfile
import urllib.request
import numpy as np
from pandas.core.common import flatten
from plotnine import *
from array import *
import scipy.stats as stats
import math
import matplotlib as mpl
import matplotlib.pyplot as plt
import statsmodels.api as sm
from sklearn import linear_model
from sklearn.metrics import mean_squared_error, r2_score
import statsmodels.formula.api as smf
import statsmodels.stats.api as sms
```

Ejemplo 2 - Parte 1

En este ejemplo se profundizará en los modelos de regresión penalizada. Cargar el dataset sobre el rendimiento estudiantil, y realizar un modelo de regresión Ridge del puntaje final con las variables restantes.

- Con el parámetro de encogimiento $\alpha=10$
- Con el parámetro de encogimiento $\alpha=100$
- Con el parámetro de encogimiento $\alpha=1000$
- Comparar los modelos con el modelo regresión lineal múltiple. ¿Qué se observa?
- Realizar la validación cruzada para determinar el valor de α que proporciona un menor error cuadrático
- Comparar el comportamiento de los coeficientes cuando α disminuye y cuando α tiende a hacerse más grande.

```
In [ ]: datos = pd.read_csv("ultimate_student_productivity_dataset_5000.csv")
datos
```

```
In [ ]: datos.info()
```

```
In [ ]: respuesta = datos["exam_score"].copy()
predictoras = datos.drop(["student_id", "gender", "academic_level", "internet_qu
```

```
In [ ]: from sklearn.preprocessing import StandardScaler
        scaler = StandardScaler()
        scaler.fit(predictoras)

        predictoras_est = pd.DataFrame(scaler.transform(predictoras), columns = predicto
```

```
In [ ]: from sklearn.model_selection import train_test_split

#Obtenemos las muestras de entrenamiento y test por medio de la función train_te
X_train, X_test, y_train, y_test= train_test_split(predictoras_est, respuesta, t
```

```
In [ ]: #Importar clase
        from sklearn.linear_model import Ridge

        #Ajustar el modelo
        ridge_reg = Ridge(alpha = 1, solver = "auto")
        ridge_reg.fit(X_train, y_train)

        #Obtener coeficientes del modelo
        #Intercepto
        print("El intercepto que se tiene es %s" %ridge_reg.intercept_)
        #Coeficientes de regresion
        coefs_ridge_a = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":
        coefs_ridge_a
```

```
In [ ]: #Ajustar el modelo
        ridge_reg_b = Ridge(alpha = 10, solver = "auto")
        ridge_reg_b.fit(X_train, y_train)

        #Obtener coeficientes del modelo

        #Intercepto
        print("El intercepto que se tiene es %s" %ridge_reg_b.intercept_)
        #Coeficientes de regresion
        coefs_ridge_b = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":
        coefs_ridge_b
```

```
In [ ]: #Ajustar el modelo
        ridge_reg_c = Ridge(alpha = 100, solver = "auto")
        ridge_reg_c.fit(X_train, y_train)

        #Obtener coeficientes del modelo

        #Intercepto
        print("El intercepto que se tiene es %s" %ridge_reg_c.intercept_)
        #Coeficientes de regresion
        coefs_ridge_c = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":
        coefs_ridge_c
```

```
In [ ]: # Definir matriz de diseño y variable respuesta

        #Añadir intercepto
        X2 = sm.add_constant(predictoras_est)

        #Ajustar el modelo
        est = sm.OLS(respuesta, X2)

        #Ver ajuste
```

```
est2 = est.fit()
print(est2.summary())
```

```
In [ ]: # Variance Inflation Factor
from statsmodels.stats.outliers_influence import variance_inflation_factor
#Crear dataframe para aplicar VIF

VIF = pd.DataFrame()
VIF["variable"] = predictoras.columns

#Aplicar el método
VIF["VIF"] = [variance_inflation_factor(predictoras.values, i) for i in range(len(predictoras.columns))]

VIF
```

```
In [ ]: #Importar clase
from sklearn.linear_model import LinearRegression

#Ajustar el modelo
lm = LinearRegression()
lm.fit(predictoras_est, respuesta)

#Obtener error para cada uno
y_pred_lm = lm.predict(X_test)
y_pred_ridge = ridge_reg.predict(X_test)
y_pred_ridge_b = ridge_reg_b.predict(X_test)
y_pred_ridge_c = ridge_reg_c.predict(X_test)

mse_lm = mean_squared_error(y_test, y_pred_lm)
mse_ridge = mean_squared_error(y_test, y_pred_ridge)
mse_ridge_b = mean_squared_error(y_test, y_pred_ridge_b)
mse_ridge_c = mean_squared_error(y_test, y_pred_ridge_c)
errores_ridge = [mse_lm, mse_ridge, mse_ridge_b, mse_ridge_c]

comparación_ridge = pd.DataFrame({"Modelo": ["Lineal", r"Ridge $\alpha=1$", r"Ridge $\alpha=10$"],
                                  "MSE": errores_ridge})

comparación_ridge
```

El modelo de regresión lineal presenta el menor MSE con 23.7884, lo que indica que es el modelo con mejor capacidad predictiva en este caso. Los modelos Ridge con $\alpha=1$ y $\alpha=10$ tienen MSE = 23.95, muestran valores muy cercanos al modelo lineal, lo que sugiere que la regularización aplicada es leve y no altera significativamente el ajuste del modelo.

El modelo Ridge con MSE = 24.0791 presenta un ligero incremento en el error, indicando que una mayor penalización.

```
In [ ]: fig = plt.figure(figsize=(10, 6))

#Ancho de barra
barWidth = 0.15

#Definir posición barras series
r1 = np.arange(len(lm.coef_[0:8]))
r2 = [x + barWidth for x in r1]
r3 = [x + barWidth for x in r2]
```

```

r4 = [x + barWidth for x in r3]

#Pintar las barras
plt.bar(r1, lm.coef_[0:8], color = "blue", width = barWidth, edgecolor = "white")
plt.bar(r2, ridge_reg.coef_[0:8], color = "red", width = barWidth, edgecolor = "w
plt.bar(r3, ridge_reg_b.coef_[0:8], color = "green", width = barWidth, edgecolor
plt.bar(r4, ridge_reg_c.coef_[0:8], color = "black", width = barWidth, edgecolor

plt.xticks([r + barWidth for r in range(len(lm.coef_[0:8]))],
            [predictoras.columns[i] for i in range(len(predictoras.columns[0:8]))])
plt.legend();
plt.xlabel("Variable");
plt.ylabel("Beta");
plt.show(fig)

```

```

In [ ]: fig2 = plt.figure(figsize=(10, 6))

#Ancho de barra
barWidth = 0.15

#Definir posicion barras series
r1 = np.arange(len(lm.coef_[8:]))
r2 = [x + barWidth for x in r1]
r3 = [x + barWidth for x in r2]
r4 = [x + barWidth for x in r3]

#Pintar las barras
plt.bar(r1, lm.coef_[8:], color = "blue", width = barWidth, edgecolor = "white",
plt.bar(r2, ridge_reg.coef_[8:], color = "red", width = barWidth, edgecolor = "w
plt.bar(r3, ridge_reg_b.coef_[8:], color = "green", width = barWidth, edgecolor
plt.bar(r4, ridge_reg_c.coef_[8:], color = "black", width = barWidth, edgecolor

plt.xticks([r + barWidth for r in range(len(lm.coef_[8:]))],
            [predictoras.columns[i] for i in range(8,len(predictoras.columns))]), r
plt.legend();
plt.xlabel("Variable");
plt.ylabel("Beta");
plt.show(fig2)

```

```

In [ ]: from sklearn.linear_model import RidgeCV
from sklearn.model_selection import KFold, cross_val_score

alphas = np.logspace(-3, 4, 100)
ridge = RidgeCV(alphas=alphas, cv=10)
ridge.fit(X_train, y_train)
kf = KFold(n_splits=10, shuffle=True, random_state=2026)

mse_mean = list()
mse_std = list()

for alpha in alphas:
    modelo_tmp = RidgeCV(alphas=[alpha], cv=None)
    scores = cross_val_score(modelo_tmp, X_train, y_train, scoring='neg_mean_squ

    mse = -scores
    mse_mean.append(mse.mean())
    mse_std.append(mse.std())

mse_mean = np.array(mse_mean)

```

```

mse_std = np.array(mse_std)

alpha_min_ridge = ridge.alpha_

idx_min = np.argmin(mse_mean)
mse_min = mse_mean[idx_min]
se_min = mse_std[idx_min]

idx_1se = np.where(mse_mean <= mse_min + se_min)[0][0]
alpha_1se = alphas[idx_1se]

#Realizar el gráfico
x = np.log(alphas)

plt.figure(figsize=(8, 4))
plt.errorbar(x, mse_mean, yerr=mse_std, fmt='o', color='red', ecolor='gray', eli
plt.axvline(np.log(alpha_min_ridge), linestyle='--', color="gray")

plt.xlabel("log(alpha)")
plt.ylabel('Mean Squared Error')
plt.title('Validación Cruzada para Ridge')
plt.gca().invert_xaxis()

plt.tight_layout()
plt.show()

print("El valor de alpha que minimiza el mse es %s" %alpha_min_ridge)

```

```

In [ ]: #Ajustar el modelo
ridge_reg_d = Ridge(alpha = alpha_min_ridge)
ridge_reg_d.fit(X_train, y_train)

```

```

In [ ]: y_pred_ridge_d = ridge_reg_d.predict(X_test)
mse_ridge_d = mean_squared_error(y_test, y_pred_ridge_d)
mse_ridge_d

```

```

In [ ]: coefs = list()
for a in alphas:
    ridge = linear_model.Ridge(alpha=a, fit_intercept=False)
    ridge.fit(predictoras_est, respuesta)
    coefs.append(ridge.coef_)

ax = plt.gca()

ax.plot(alphas, coefs)
ax.set_xscale("log")
ax.set_xlim(ax.get_xlim()[::-1]) # reverse axis
plt.xlabel(r"$\alpha$")
plt.ylabel("Coeficientes")
plt.title("Evolución de los coeficientes")
plt.axis("tight")
plt.legend([f"Feature {i + 1}" for i in range(predictoras_est.shape[1])], loc="b
plt.legend([predictoras.columns[i] for i in range(predictoras_est.shape[1])], lo
plt.show()

```

Ejemplo 2 - Parte 2

Relizar un modelo de regresión de Lasso del puntaje final con las variables restantes.

- Con el parámetro de encogimiento $\alpha=0.05$
- Con el parámetro de encogimiento $\alpha=0.5$
- Con el parámetro de encogimiento $\alpha=1$
- Comparar los modelos con el modelo regresión lineal múltiple. ¿Qué se observa?
- Realizar la validación cruzada para determinar el valor de α que proporciona un menor error cuadrático
- Comparar el comportamiento de los coeficientes cuando α disminuye y cuando α tiende a hacerse más grande.

```
In [ ]: #Importar clase
from sklearn.linear_model import Lasso

#Ajustar el modelo-
lasso_reg = Lasso(alpha = 0.01)
lasso_reg.fit(X_train, y_train)

#Intercepto
print("El intercepto que se tiene es %s" %lasso_reg.intercept_)
#Coeficientes de regresion
coefs_lasso_a = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":
coefs_lasso_a

In [ ]: #Ajustar el modelo-
lasso_reg_b = Lasso(alpha = 1)
lasso_reg_b.fit(X_train, y_train)

#Obtener coeficientes del modelo

#Intercepto
print("El intercepto que se tiene es %s" %lasso_reg_b.intercept_)
#Coeficientes de regresion
coefs_lasso_b= pd.DataFrame({"Variables": predictoras.columns, "Coeficientes": 1
coefs_lasso_b

In [ ]: #Ajustar el modelo-
lasso_reg_c = Lasso(alpha = 2)
lasso_reg_c.fit(X_train, y_train)

#Obtener coeficientes del modelo

#Intercepto
print("El intercepto que se tiene es %s" %lasso_reg_c.intercept_)
#Coeficientes de regresion
coefs_lasso_c = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes":
coefs_lasso_c

In [ ]: #Obtener error para cada uno
y_pred_lasso = lasso_reg.predict(X_test)
y_pred_lasso_b = lasso_reg_b.predict(X_test)
y_pred_lasso_c = lasso_reg_c.predict(X_test)
```

```

mse_lasso = mean_squared_error(y_test, y_pred_lasso)
mse_lasso_b = mean_squared_error(y_test, y_pred_lasso_b)
mse_lasso_c = mean_squared_error(y_test, y_pred_lasso_c)
errores_lasso = [mse_lm, mse_lasso, mse_lasso_b, mse_lasso_c]

comparación_lasso = pd.DataFrame({"Modelo": ["Lineal", r"Lasso $\alpha=0.01$", r"Lasso $\alpha=0.001$"],
                                  "MSE": errores_lasso})

comparación_lasso

```

El modelo de regresión lineal presenta el menor MSE de 23.7884, lo que indica la mejor capacidad predictiva. El modelo Lasso con $\alpha=0.01$ tiene MSE = 23.9483 muestra un desempeño muy cercano al modelo lineal, lo que sugiere que una regularización leve permite mantener la precisión mientras comienza la selección de variables. Sin embargo, los modelos Lasso con MSE = 26.0094 y MSE = 31.7573 presentan incrementos importantes en el error, evidenciando que una penalización más fuerte provoca una pérdida significativa de capacidad predictiva.

```

In [ ]: from sklearn.linear_model import LassoCV, lasso_path

# Ajustar LassoCV
lasso_cv = LassoCV(cv=10, random_state=2026, n_alphas=100)
lasso_cv.fit(X_train, y_train)

alphas = lasso_cv.alphas_
mse_path = lasso_cv.mse_path_
mse_mean = mse_path.mean(axis=1)
mse_std = mse_path.std(axis=1)
alpha_min = lasso_cv.alpha_

idx_min = np.argmin(mse_mean)
mse_min = mse_mean[idx_min]
se_min = mse_std[idx_min]
idx_1se = np.where(mse_mean <= mse_min + se_min)[0][0]
alpha_1se = alphas[idx_1se]

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
alphas_path, coefs_path, _ = lasso_path(X_scaled, y_train, alphas=alphas)
n_nonzero = np.sum(coefs_path != 0, axis=0)

x = np.log(alphas)

fig, ax = plt.subplots(figsize=(8, 4))

plt.errorbar(x, mse_mean, yerr=mse_std, fmt='o', color='red', ecolor='gray', eli

ax.set_xlabel('log(alpha)')
ax.set_ylabel('Mean-Squared Error')
ax.set_title('Lasso Cross-Validation')
ax.invert_xaxis()

plt.axvline(np.log(alpha_min), linestyle='--', color="gray")
plt.tight_layout()

```

```
plt.show()

print("alpha_min =", alpha_min)
```

```
In [ ]: fig = plt.figure(figsize=(10, 6))

#Ancho de barra
barWidth = 0.15

#Definir posicion barras series
r1 = np.arange(len(lm.coef_[0:8]))
r2 = [x + barWidth for x in r1]
r3 = [x + barWidth for x in r2]
r4 = [x + barWidth for x in r3]

#Pintar Las barras
plt.bar(r1, lm.coef_[0:8], color = "blue", width = barWidth, edgecolor = "white")
plt.bar(r2, lasso_reg.coef_[0:8], color = "red", width = barWidth, edgecolor = "w")
plt.bar(r3, lasso_reg_b.coef_[0:8], color = "green", width = barWidth, edgecolor = "w")
plt.bar(r4, lasso_reg_c.coef_[0:8], color = "black", width = barWidth, edgecolor = "w")

plt.xticks([r + barWidth for r in range(len(lm.coef_[0:8]))],
            [predictoras.columns[i] for i in range(len(predictoras.columns[0:8]))])
plt.legend();
plt.xlabel("Variable");
plt.ylabel("Beta");
plt.show(fig)
```

```
In [ ]: fig2 = plt.figure(figsize=(10, 6))

#Ancho de barra
barWidth = 0.15

#Definir posicion barras series
r1 = np.arange(len(lm.coef_[8:]))
r2 = [x + barWidth for x in r1]
r3 = [x + barWidth for x in r2]
r4 = [x + barWidth for x in r3]

#Pintar Las barras
plt.bar(r1, lm.coef_[8:], color = "blue", width = barWidth, edgecolor = "white", r
plt.bar(r2, lasso_reg.coef_[8:], color = "red", width = barWidth, edgecolor = "w
plt.bar(r3, lasso_reg_b.coef_[8:], color = "green", width = barWidth, edgecolor
plt.bar(r4, lasso_reg_c.coef_[8:], color = "black", width = barWidth, edgecolor

plt.xticks([r + barWidth for r in range(len(lm.coef_[8:]))],
            [predictoras.columns[i] for i in range(8,len(predictoras.columns))]), r
plt.legend();
plt.xlabel("Variable");
plt.ylabel("Beta");
plt.show(fig2)
```

```
In [ ]: #Importar clase
from sklearn.linear_model import ElasticNetCV

#Ajustar el modelo
e_net = ElasticNetCV(cv = 10, l1_ratio = 0.1, random_state=2026)
e_net.fit(X_train, y_train)
```



```
#Obtener coeficientes del modelo

#Intercepto
print("El intercepto que se tiene es %s" %e_net.intercept_)
#Coeficientes de regresion
coefs_e_net = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes": e_net.coefs_e_net
```

Ejemplo 2 - Parte 3

Realizar un modelo de Red elástica del puntaje final con las variables restantes.

- Fijando $r=0.1$
- Fijando $r=0.90$ (En ambos casos ajustar por validación cruzada con $k=10$ particiones)
- Comparar los modelos con el modelo regresión lineal múltiple. ¿Qué se observa?
- Realizar la validación cruzada para determinar el valor de α y r que proporciona un menor error cuadrático.

```
In [ ]: #Ajustar el modelo
e_net_b = ElasticNetCV(cv = 10, l1_ratio = 0.9, random_state=2026)
e_net_b.fit(X_train, y_train)

#Obtener coeficientes del modelo

#Intercepto
print("El intercepto que se tiene es %s" %e_net_b.intercept_)
#Coeficientes de regresion
coefs_e_net_b = pd.DataFrame({"Variables": predictoras.columns, "Coeficientes": e_net_b.coefs_e_net_b
```

```
In [ ]: #Obtener error para cada uno
y_pred_e_net = e_net.predict(X_test)
y_pred_e_net_b = e_net_b.predict(X_test)

mse_e_net = mean_squared_error(y_test, y_pred_e_net)
mse_e_net_b = mean_squared_error(y_test, y_pred_e_net_b)

errores_e_net = [mse_lm, mse_e_net, mse_e_net_b]

comparación_e_net= pd.DataFrame({"Modelo": ["Lineal", r"Red elástica $r=0.1$", r"Red elástica $r=0.90$"],
                                "MSE": errores_e_net})

comparación_e_net
```

El modelo de Red Elástica con $r=0.9$ tiene MSE = 23.9269 muestra un desempeño muy cercano al modelo lineal, lo que sugiere que una combinación adecuada de penalización y permite mantener la precisión del modelo mientras controla su complejidad.

El modelo de Red Elástica con $r=0.1$ posee MSE = 24.6271 presenta un incremento en el error, indicando que una mayor regularización produce una ligera pérdida de capacidad predictiva.

```
In [ ]: modelo_cv = ElasticNetCV(alphas=alphas,l1_ratio=np.linspace(0.1, 0.99, 10),
                                cv=10, max_iter=1000, random_state=2026)

modelo_cv.fit(predictoras_est, respuesta)

print("El valor de alfa que minimiza el error es %s" %modelo_cv.alpha_)
print("El valor de r que minimiza el error es %s" %modelo_cv.l1_ratio_)
```

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt

from sklearn.linear_model import ElasticNet, ElasticNetCV
from sklearn.model_selection import KFold, cross_val_score

# Parámetros
r = 0.99

# Modelo CV
enet_cv = ElasticNetCV( alphas=alphas,l1_ratio=r,cv=10, max_iter=1000, random_st
enet_cv.fit(predictoras_est, respuesta)
kf = KFold(n_splits=10, shuffle=True, random_state=2026)

mse_mean = list()
mse_std = list()

# Validación manual
for alpha in alphas:
    modelo = ElasticNet(alpha=alpha, l1_ratio=r, max_iter=1000)
    scores = cross_val_score(modelo,predictoras_est,respuesta,scoring='neg_mean_
    mse = -scores
    mse_mean.append(mse.mean())
    mse_std.append(mse.std())

mse_mean = np.array(mse_mean)
mse_std = np.array(mse_std)

alpha_min = enet_cv.alpha_

idx_min = np.argmin(mse_mean)
mse_min = mse_mean[idx_min]
se_min = mse_std[idx_min]

idx_1se = np.where(mse_mean <= mse_min + se_min)[0][-1]
alpha_1se = alphas[idx_1se]

x = np.log(alphas)

plt.figure(figsize=(8, 4))

plt.errorbar( x, mse_mean, yerr=mse_std, fmt='o', ecolor='gray',elinewidth=0.8,

plt.axvline(np.log(alpha_min), linestyle='--', color='gray')

plt.xlabel("log(alpha)")
plt.ylabel("Mean Squared Error")
plt.title("Validación cruzada red elástica")
plt.gca().invert_xaxis()
```

```
plt.tight_layout()
plt.show()

print("alfa_min = ", alpha_min)
```

```
In [ ]: #Ajustar el modelo
e_net_c = ElasticNet(alpha=modelo_cv.alpha_, l1_ratio = modelo_cv.l1_ratio_, ran
e_net_c.fit(X_train, y_train)

y_pred_e_net_c = e_net_c.predict(X_test)
mse_e_net_c = mean_squared_error(y_test, y_pred_e_net_c)

print("El error cuadrático que se tiene es de %s" %mse_e_net_c)
```